

# Topic Selection and Problem Definition

## Topic Chosen – Topic 2: AI for Design Verification (Google Track)

### Problem Description –

- The goal of this project is to build an AI-powered agent that can automatically generate Verilog testbenches for hardware design verification.
- In chip design, verification is one of the most time-consuming and costly stages, often accounting for up to 70% of the total design effort.
- Manually writing testbenches requires significant time, a deep understanding of the design specification, and considerable engineering effort.
- This project explores whether an LLM-based agent can automate this process through an iterative workflow by generating an initial testbench, analysing simulation feedback, and incrementally refining the solution until it correctly distinguishes the correct design from the buggy ones.
- Specifically, given a natural language specification describing the intended behaviour of a hardware module, along with multiple RTL implementations (as Verilog files), where only one is functionally correct and the rest contain bugs, the agent must generate a Verilog testbench that:
  - Passes (all assertions/checks succeed) when simulated against the single correct RTL implementation
  - Fails (at least one assertion/check fails) when simulated against remaining incorrect/buggy RTL implementations
- In short, the generated testbench must identify and be capable of distinguishing the correct design from all the remaining buggy variants.

### Required Inputs –

- Natural Language Specification – A textual description of the expected behaviour of the hardware module.
- A set of Verilog source files (RTL implementation), where:
  - Exactly one implementation is functionally correct
  - The remaining implementations contain one or more functional bugs
- Port names, widths, clock/reset conventions, and any relevant signal-level details (either extracted from the RTL or provided in the specification).

### Output Generated –

- Verilog Testbench File – A complete, self-contained Verilog testbench that:
  - Instantiates the design under test (DUT)
  - Generates clock and reset signals
  - Applies a sequence of input stimulus vectors
  - Contains assertions or checker logic (e.g., if-else checks, `$display/$error` statements) to verify expected outputs
  - Produces a clear PASS/FAIL verdict upon simulation
- Simulation Results / Verdict Log – For each RTL implementation, the output of running the testbench via iverilog indicating:
  - PASS for the correct implementation
  - FAIL for all the remaining buggy implementations

## Evaluation Method –

- Discriminative Accuracy (Primary Metric)
  - The testbench must PASS on the correct RTL implementation
  - The testbench must FAIL on every incorrect RTL implementation
- Compilation Success Rate
  - The generated testbench must compile without errors using iverilog
  - Measures the syntactic correctness of the LLM-generated Verilog code
- Simulation Validity
  - The testbench must simulate successfully (iverilog simulation runtime)
  - No hangs, infinite loops, or runtime errors
- Test Coverage Quality
  - Evaluate whether the testbench covers:
  - Corner cases (e.g., boundary values, overflow, reset behaviour)
  - Normal operation scenarios
  - Edge-triggered behaviour
- Iterative Improvement, if the agent employs an iterative refinement loop (generate → simulate → analyse errors → regenerate), evaluate:
  - Number of iterations needed to achieve full discrimination
  - Whether the agent successfully self-corrects based on simulation feedback
- Generalizability
  - Test the agent across multiple design types (e.g., combinational logic, sequential circuits, FSMs) to assess how well the approach generalizes beyond a single problem instance